https://cp-algorithms.com/algebra/bit-manipulation.html

Manipulación de bits

Número binario

Un número binario es un número expresado en el sistema de numeración de base 2 o sistema de numeración binario, es un método de expresión matemática que utiliza solo dos símbolos: típicamente "0" (cero) y "1" (uno).
Decimos que un determinado bit está **activo** si es uno y **se borra** si es cero.
El número binario $(a_k a_{k-1} \dots a_1 a_0)_2$ representa el número:
$(a_k a_{k-1} \dots a_1 a_0)_2 = a_k \cdot 2^k + a_{k-1} \cdot 2^{k-1} + \dots + a_1 \cdot 2^1 + a_0 \cdot 2^0$.

Por ejemplo, el número binario 11012 representa el número 13:
$1101_2 = 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$
$= 1 \cdot 8 + 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 = 13$
Las computadoras representan los números enteros como números binarios. Los números enteros positivos (con y sin signo) se representan simplemente con sus dígitos binarios, y los números negativos con signo (que pueden ser positivos y negativos) suelen representarse con el complemento a dos .

```
unsigned int unsigned_number = 13;
assert(unsigned_number == 0b1101);

int positive_signed_number = 13;
assert(positive_signed_number == 0b1101);

int negative_signed_number = -13;
assert(negative_signed_number == 0b1111'1111'1111'1111'1111'1111'1111'0011);
```

Las CPU manipulan esos bits con gran rapidez mediante operaciones específicas. En algunos problemas, podemos aprovechar estas representaciones numéricas binarias y acelerar el tiempo de ejecución. Para otros problemas (normalmente en combinatoria o programación dinámica), donde queremos rastrear qué objetos ya seleccionamos de un conjunto dado, podemos usar un entero suficientemente grande donde cada dígito representa un objeto y, según si seleccionamos o eliminamos el objeto, activamos o desactivamos el dígito.

Operadores de bits

Todos esos operadores introducidos son instantáneos (a la misma velocidad que una suma) en una CPU para números enteros de longitud fija.

Operadores bit a bit

& El operador AND bit a bit compara cada bit de su primer operando con el bit correspondiente de su segundo operando. Si ambos bits son 1, el bit de resultado correspondiente se establece en 1. De lo contrario, el bit de resultado correspondiente se establece en 0.

| El operador OR inclusivo bit a bit compara cada bit de su primer operando con el bit correspondiente de su segundo operando. Si uno de los dos bits es 1, el bit de resultado correspondiente se establece en 1. De lo contrario, el bit de resultado correspondiente se establece en 0.

^ El operador OR exclusivo (XOR) a nivel de bits compara cada bit de su primer operando con el bit correspondiente de su segundo operando. Si un bit es 0 y el otro es 1, el bit de resultado correspondiente se establece en 1. De lo contrario, el bit de resultado correspondiente se establece en 0.

~ El operador de complemento bit a bit (NOT) invierte cada bit de un número; si un bit está configurado, el operador lo borrará; si está configurado, el operador lo establecerá.

n = 01011000
n-1 = 01010111
---
n & (n-1) = 01010000

n = 01011000
n-1 = 01010111
---
n | (n-1) = 01011111

n = 01011000
n-1 = 01010111
---
n ^ (n-1) = 00001111

n = 01011000

~n = 10100111

Operadores de desplazamiento

Hay dos operadores para desplazar bits.

Desplazamiento a la derecha (>>): Desplaza un número a la derecha eliminando los últimos dígitos binarios del número. Cada desplazamiento de uno representa una división entera entre 2, por lo que un desplazamiento a la derecha de k representa una división entera por 2^k.

Ejemplo:

$5 \gg 2 = 101_2 \gg 2 = 1_2 = 1$ Lo que es equivalente a: $\frac{5}{2^2} = \frac{5}{4} = 1$ Sin embargo, para una computadora, mover bits es mucho más rápido que hacer divisiones.

Desplazamiento a la izquierda (<<): Desplaza un número a la izquierda añadiendo ceros a la derecha. De forma similar al desplazamiento a la derecha por k, un desplazamiento a la izquierda por k representa una multiplicación por 2^k.

Ejemplo:

$5 \ll 3 = 101_2 \ll 3 = 101000_2 = 40$ Lo que es equivalente a: $5 \cdot 2^3 = 5 \cdot 8 = 40$

Sin embargo, tenga en cuenta que para un entero de longitud fija eso significa eliminar los dígitos más a la izquierda y, si cambia demasiado, terminará con el número 0

Trucos útiles

Establecer / borrar / alternar un bit

Usando desplazamientos bit a bit y algunas operaciones bit a bit básicas podemos fácilmente establecer, invertir o borrar un bit. **1 << x** es un número con solo el x-ésimo bit establecido, mientras usando **~ (1 << x)** es un número con todos los bits establecidos excepto el x-ésimo bit

n | (1 << x) Establece (pone en 1) el bit número *x* en el número n.Si el bit ya estaba en 1, no cambia nada.

n ^ (1 << x) Invierte el bit número *x* en el número n. Si el bit estaba en 0 pasa a 1, y si estaba en 1 pasa a 0.

n & ~ (1 << x) Limpia (pone en 0) el bit número *x* en el número n. Sin importar su valor inicial, después de esta operación ese bit será 0.

Comprobar si un bit está establecido

El valor de la **x** El bit -ésimo se puede comprobar desplazando el número **x** posiciones a la derecha, de modo que El bit -ésimo está en el lugar de la unidad, después de lo cual podemos extraerlo realizando un **&** bit a bit con **1**.

```
bool is_set(unsigned int number, int x) {
   return (number >> x) & 1;
}
```

Comprobar divisibilidad por potencia de 2

Usando la operación **y**, podemos comprobar si un número **n** es **par** porque n & 1 = 0, si n es par, y n & 1 = 1 si n es **impar**. En términos más generales, n es divisible por 2^k exactamente cuando n & ($2^k - 1$) = 0.

```
bool isDivisibleByPowerOf2(int n, int k) {
   int powerOf2 = 1 << k;
   return (n & (powerOf2 - 1)) == 0;
}
```

Podemos calcular 2^k desplazando 1 hacia la izquierda k posiciones. El truco funciona, porque $2^k - 1$ es un número que consta exactamente de k unos. Y un número que es divisible por 2^k debe tener cero dígitos en esos lugares.

Comprobar si un número es potencia de 2

Una potencia de dos es un número que solo tiene un bit (por ejemplo, $32 = 0010\ 0000_2$), mientras que el predecesor de ese número tiene ese bit no establecido y todos los bits posteriores establecidos ($31 = 0001\ 1111_2$). Por lo tanto, la operación AND bit a bit de un número con su predecesor siempre será 0, ya que no tienen bits en común. Se puede comprobar fácilmente que esto solo ocurre para las potencias de dos y para el número 0, que ya no tiene ningún bit establecido.

```
bool isPowerOfTwo(unsigned int n) {

   return n && !(n & (n - 1));

}
```

Quitar el bit activo más a la derecha

La expresión **n&(n−1)** se puede utilizar para desactivar el bit establecido más a la derecha de un número **n**. Esto funciona porque la expresión **n−1** invierte todos los bits después del bit establecido más a la derecha de **n**, incluyendo ese bit establecido. Por lo tanto, todos esos bits son diferentes del número original, y al realizar una operación **AND** bit a bit, se ponen en **0**, dando como resultado el número original **n** con el bit establecido más a la derecha apagado.

Por ejemplo, considere el número 52=0011 0100_2
n = 00110100
n-1 = 00110011
---
n & (n-1) = 00110000

Algoritmo de Brian Kernighan

Podemos contar el número de bits establecidos con la expresión anterior.
La idea es considerar solo los bits establecidos de un entero desactivando su bit establecido más a la derecha (después de contarlo), de modo que la siguiente iteración del bucle considere el siguiente bit más a la derecha.

```
int countSetBits(int n)
{
   int count = 0;
   while (n)
   {
      n = n & (n - 1);
      count++;
   }
   return count;
}
```

Contar bits establecidos hasta n

Para contar el número de bits establecidos de todos los números hasta el número n (inclusive), podemos ejecutar el algoritmo de Brian Kernighan en todos los números hasta n. Pero esto resultará en un "Límite de tiempo excedido" en las presentaciones del concurso.
Podemos utilizar el hecho de que para números hasta 2^x (es decir, de 1 a 2^x-1) hay $x \cdot 2^{x-1}$ bits establecidos. Esto se puede visualizar de la siguiente manera.
0 -> 0 0 0 0
1 -> 0 0 0 1
2 -> 0 0 1 0
3 -> 0 0 1 1
4 -> 0 1 0 0
5 -> 0 1 0 1
6 -> 0 1 1 0
7 -> 0 1 1 1
8 -> 1 0 0 0
Podemos ver que todas las columnas excepto la más a la izquierda tienen 4 (es decir 2^2) establece bits cada uno, es decir, hasta el número 2^3 - 1, el número de bits establecidos es $3*2^{3-1}$.
Con los nuevos conocimientos en mano podemos elaborar el siguiente algoritmo:

- Encuentra la potencia más alta de 2 que sea menor o igual que el número dado. Sea este número x.
- Calcular el número de bits establecidos a partir de 1 a 2^x - 1 utilizando la fórmula $x*2^{x-1}$.
- Cuente el número de bits establecidos en el bit más significativo de 2^x a n y agregarlo.
- Sustraer 2^x y repita los pasos anteriores utilizando el nuevo n.

```
int countSetBits(int n) {
      int count = 0;
      while (n > 0) {
         int x = std::bit_width(n) - 1;
         count += x << (x - 1);
         n -= 1 << x;
```

```
        count += n + 1;
    }
    return count;
}
```

Trucos adicionales

n&(n+1) Borrar todos los bits a la derecha(todos los final) del último bit establecido
Ejemplo en binario: $0011\ 0111_2 \rightarrow 0011\ 0000_2$
Esto limpia todos los bits menos significativos al último bit "1" de n. Es útil cuando quieres eliminar la "cola" de bits activos.
n | (n+1) Establecer el último bit borrado
Ejemplo en binario: $0011\ 0101_2 \rightarrow 0011\ 0111_2$
Esto vuelve a poner a 1 el bit inmediatamente después del último bit activo. Sirve para extender la secuencia de unos a la derecha del último bit activo.
n & −n Extraer el último bit establecido
Ejemplo en binario: $0011\ 0110_2 \rightarrow 0000\ 0010_2$
Esto obtiene el bit menos significativo que está a 1 en n. Es muy útil para recorrer o contar bits activos en algoritmos de manipulación de bits.

Compatibilidad con lenguajes y compiladores

C++ admite algunas de esas operaciones desde C++20 a través de la biblioteca estándar bit :

- has_single_bit: comprueba si el número es una potencia de dos
- bit_ceil/ bit_floor: redondear hacia arriba o hacia abajo a la siguiente potencia de dos
- rotl/ rotr: rotar los bits en el número
- countl_zero/ countr_zero/ countl_one/ countr_one: cuenta los ceros y unos iniciales y finales
- popcount: cuenta el número de bits establecidos

Además, algunos compiladores incluyen funciones predefinidas que facilitan el trabajo con bits. Por ejemplo, GCC define una lista en las funciones integradas proporcionadas por GCC que también funcionan en versiones anteriores de C++:

- __builtin_popcount(unsigned int) devuelve el número de bits establecidos (__builtin_popcount(0b0001'0010'1100) == 4)
- __builtin_ffs(int) encuentra el índice del primer bit establecido (más a la derecha) (__builtin_ffs(0b0001'0010'1100) == 3)
- __builtin_clz(unsigned int) el recuento de ceros a la izquierda (__builtin_clz(0b0001'0010'1100) == 23)
- __builtin_ctz(unsigned int) el recuento de ceros finales (__builtin_ctz(0b0001'0010'1100) == 2)
- __builtin_parity(x) la paridad (par o impar) del número de unos en la representación de bits

Tenga en cuenta que algunas de las operaciones (tanto las funciones de C++20 como las integradas del compilador) pueden ser bastante lentas en GCC si no habilita un objetivo de compilador específico con #pragma GCC target("popcnt").